Análise de Dependências Funcionais e Formas Normais

Beatriz Pontes Camargo

1 Introdução

Este trabalho apresenta o desenvolvimento do `fdtool`, uma ferramenta de linha de comando implementada em linguagem C para a análise de dependências funcionais em esquemas relacionais. O programa permite realizar operações importantes para o estudo da teoria de normalização de bancos de dados, auxiliando na identificação de propriedades dos esquemas e na verificação de suas formas normais.

A ferramenta implementa quatro operações: cálculo do fecho de um conjunto de atributos, obtenção de uma cobertura mínima de dependências funcionais, identificação de todas as chaves candidatas e verificação das formas normais de Boyce-Codd (BCNF) e Terceira Forma Normal (3FN). As operações são selecionadas por argumentos na linha de comando, e as dependências funcionais são fornecidas por meio de arquivos com extensão `.fds`.

2 Definições e Algoritmos

Uma dependência funcional $X \to Y$ indica que, em um esquema relacional, os atributos de X determinam unicamente os atributos de Y. O fecho de um conjunto de atributos X, denotado por X^+, corresponde ao conjunto de atributos que podem ser determinados por X a partir das dependências funcionais. Seu cálculo é realizado inicialmente considerando os atributos de X e, em seguida, adicionando iterativamente os atributos determinados pelas dependências até que não ocorram novas inclusões.

Uma superchave é um conjunto de atributos cujo fecho contém todos os atributos do esquema. Uma chave candidata é uma superchave mínima, isto é, nenhum de seus subconjuntos próprios é capaz de determinar todo o esquema. O algoritmo de identificação das chaves enumera os subconjuntos de atributos, calcula seus fechos e seleciona aqueles que são superchaves mínimas.

A cobertura mínima é um conjunto equivalente de dependências funcionais que possui um único atributo no lado direito de cada dependência, não contém atributos desnecessários no lado esquerdo e não possui dependências redundantes. Para obtê-la, o programa separa os atributos dos lados direitos, remove os atributos estranhos dos lados esquerdos e elimina dependências redundantes, utilizando o cálculo de fechos para verificar a equivalência.

Por fim, a normalização é avaliada pelas formas normais de Boyce-Codd (BCNF) e Terceira Forma Normal (3FN). Um esquema está na BCNF quando, para toda dependência funcional não trivial $X \to A$, X é uma superchave. Na 3FN, a condição também é satisfeita quando o atributo A é primo, ou seja, pertence a alguma chave candidata. O programa calcula as chaves candidatas e verifica essas condições para identificar possíveis violações.

3 Complexidade

Sejam n o número de atributos e m o número de dependências funcionais do esquema. A complexidade das operações depende dos algoritmos utilizados, conforme resumido a seguir.

- **Fecho:** $O(nm)$. As dependências são percorridas repetidamente até que nenhum atributo novo seja adicionado, o que pode exigir até n passagens.

- **Cobertura mínima:** possui complexidade polinomial, pois realiza operações sobre as dependências e calcula fechos para verificar a remoção de atributos e dependências redundantes.
- **Chaves candidatas:** possui complexidade exponencial, pois enumera os subconjuntos dos atributos. Existem 2^n subconjuntos, e cada um pode exigir cálculos de fecho e verificações de minimalidade.
- **Formas normais:** também possui complexidade exponencial no pior caso, pois depende da identificação das chaves candidatas para determinar os atributos primos.

Como os atributos são representados por uma máscara de bits de 32 bits, as operações básicas sobre conjuntos são realizadas em tempo constante. Entretanto, a enumeração de subconjuntos torna a busca por chaves a operação de maior custo à medida que o número de atributos aumenta.

4 Estudos de Caso

Foram definidos três casos para avaliar o cálculo de fechos, a identificação de chaves candidatas e a verificação das formas normais.

4.1 Caso 1: Violação da 3FN e da BCNF

Considere o esquema $U = \{A, B, C\}$, com $F = \{A \to B,\ B \to C\}$. O comando `./fdtool closure --fds caso1.fds --X A` retorna $\{A, B, C\}$, indicando que A é chave candidata. Entretanto, $B \to C$ viola a BCNF, pois B não é superchave, e também viola a 3FN, pois C não é atributo primo.

4.2 Caso 2: 3FN, mas não BCNF

Considere $U = \{A, B, C\}$ e $F = \{AB \to C,\ C \to B\}$. As chaves candidatas são $\{A, B\}$ e $\{A, C\}$, portanto todos os atributos são primos. A dependência $C \to B$ viola a BCNF, pois C não é superchave, mas satisfaz a condição da 3FN porque B é primo. Assim, o esquema está na 3FN, mas não na BCNF. O comando `./fdtool normalform --fds caso2.fds` deve indicar `BCNF: VIOLATIONS` e `3NF: OK`.

4.3 Caso 3: Esquema na BCNF

Considere $U = \{A, B, C\}$ e $F = \{A \to B,\ A \to C\}$. A chave candidata é $\{A\}$ e todas as dependências não triviais têm uma superchave no lado esquerdo. Portanto, o esquema está na BCNF e também na 3FN. O comando `./fdtool normalform --fds caso3.fds` deve indicar `BCNF: OK` e `3NF: OK`.

5 Limitações e Decisões de Implementação

Os conjuntos de atributos são representados por máscaras de bits de 32 bits, permitindo realizar operações como união e verificação de subconjuntos de forma eficiente. A implementação considera atributos identificados por letras maiúsculas de A a Z. As dependências funcionais são lidas de arquivos `.fds`, que definem o conjunto de atributos do esquema e suas dependências.

Para calcular a cobertura mínima, as dependências são transformadas para que cada lado direito contenha apenas um atributo. Em seguida, são removidos atributos desnecessários dos lados esquerdos e dependências redundantes, utilizando o cálculo de fechos para verificar essas operações. A identificação das chaves candidatas é feita por enumeração de subconjuntos, o que simplifica o algoritmo, mas apresenta custo exponencial conforme aumenta o número de atributos.

Como limitações, o programa trabalha apenas com atributos de A a Z, e a análise das formas normais considera as dependências funcionais explicitamente informadas no arquivo de entrada. Além disso, a leitura depende do formato esperado para as linhas que definem os conjuntos de atributos e as dependências. Essas escolhas mantêm a implementação simples, mas limitam sua aplicação a entradas que respeitem o formato previsto e podem afetar a identificação de violações de formas normais quando dependências implícitas não são verificadas.

6 Conclusão

O desenvolvimento do `fdtool` permitiu implementar, em linguagem C, operações fundamentais para a análise de dependências funcionais e normalização de esquemas relacionais. A ferramenta realiza o cálculo de fechos, a obtenção de coberturas mínimas, a identificação de chaves candidatas e a verificação das formas normais de Boyce-Codd (BCNF) e Terceira Forma Normal (3FN).

Os testes realizados demonstraram o funcionamento das operações em diferentes cenários, incluindo um esquema que viola ambas as formas normais, outro que está na 3FN, mas não na BCNF, e um terceiro que satisfaz as duas condições. Dessa forma, o trabalho permitiu aplicar conceitos teóricos de bancos de dados e compreender sua implementação por meio de algoritmos e estruturas de dados em C.

Referências

[1] DOMINICO, Simone. *Banco de dados: Dependências funcionais – atualizado*. Material didático disponibilizado na UFPR Virtual, Universidade Federal do Paraná.

[2] DOMINICO, Simone. *Banco de dados: Normalização*. Material didático disponibilizado na UFPR Virtual, Universidade Federal do Paraná.